

Soil Farming Agent – Low Level Design (LLD)

1. Introduction

The **Soil Farming Agent** is a web-based Smart Agriculture Information System designed to help farmers access soil-related information and verified distributors, while allowing administrators to manage data securely. The system uses **HTML, CSS, JavaScript, and Firebase (Authentication + Realtime Database)**.

2. Objectives

- Provide farmers with accurate soil information and suitable crops
 - Allow admins to manage soil and distributor data
 - Ensure secure authentication and role-based access
 - Use real-time updates with Firebase Realtime Database
-

3. System Architecture Overview

3.1 Architecture Type

- **Client-Server Architecture (Cloud-based)**

3.2 Components

- **Frontend:** HTML, CSS, JavaScript
- **Backend (Cloud):** Firebase
- Firebase Authentication
- Firebase Realtime Database

3.3 Architecture Flow

1. User/Admin opens web application
 2. Firebase Authentication validates credentials
 3. Based on role:
 4. Admin → Admin Dashboard
 5. User → Farmer Portal
 6. Data fetched/stored in Firebase Realtime Database
-

4. Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, JavaScript
Backend	Firebase
Database	Firebase Realtime Database
Authentication	Firebase Auth (Email/Password)
Hosting	Firebase Hosting (Optional)

5. User Roles & Permissions

5.1 Admin

- Login/Register
- Add soil details
- Add distributor details
- View all data
- Logout

5.2 Farmer (User)

- Login/Register
 - View soil information
 - View distributors
 - Search soil types
 - Logout
-

6. Functional Modules (LLD)

6.1 Authentication Module

Purpose: Secure login and registration

Functions Used: - `signInWithEmailAndPassword()` - `createUserWithEmailAndPassword()` - `signOut()`

Inputs: Email, Password

Outputs: Authenticated user session

6.2 Admin Dashboard Module

6.2.1 Soil Management

Database Node: `/soils`

Data Structure:

```
soils
├── soilId
│   ├── name
│   ├── traits
│   └── crops
```

Functions: - `saveSoil()` - `db.ref('soils').push()`

6.2.2 Distributor Management

Database Node: `/distributors`

Data Structure:

```
distributors
├── distributorId
│   ├── name
│   └── contact
```

Functions: - `saveDistributor()` - `db.ref('distributors').push()`

6.3 Farmer Dashboard Module

6.3.1 View Soil Information

- Fetches real-time soil data
- Displays soil name, traits, crops

Function: `loadUserData()`

6.3.2 Search Soil Module

Purpose: Filter soils by name

Function: `searchSoils()`

Logic: - Converts input to lowercase - Matches against soil titles - Dynamically hides/shows cards

6.4 Database Seeding Module

Purpose: Initial data population

Functions: - `seedData()` - `pushInitialData()`

Used For: - Demo - Testing - Initial deployment

7. Firebase Realtime Database Design

7.1 Root Structure

```
root
├─ soils
├─ distributors
```

7.2 Security Rules (Suggested)

```
{
  "rules": {
    "soils": {
      ".read": true,
      ".write": "auth != null"
    },
    "distributors": {
      ".read": true,
      ".write": "auth != null"
    }
  }
}
```

8. UI Design Description

8.1 Pages

- Login/Register Page
- Admin Dashboard
- Farmer Dashboard

8.2 UI Components

- Cards for soil and distributor display
 - Search input box
 - Buttons for actions
 - Responsive grid layout
-

9. Error Handling & Validation

- Empty field checks
 - Firebase authentication error alerts
 - Database error handling using try-catch
-

10. Logging Strategy

- JavaScript `console.log()` for:
 - Login success/failure
 - Data upload success
 - Database fetch events
-

11. Non-Functional Requirements

- **Performance:** Real-time updates
 - **Scalability:** Firebase auto-scales
 - **Security:** Auth-based access
 - **Usability:** Simple farmer-friendly UI
 - **Availability:** Cloud-based
-

12. Future Enhancements

- Role-based admin validation
- Crop recommendation engine
- Multilingual support

- Weather API integration
 - Mobile app version
-

13. Conclusion

The Soil Farming Agent system provides a secure, scalable, and user-friendly platform for smart agriculture. This Low Level Design ensures clarity in implementation, maintainability, and future expansion.

 **Document Type:** Low Level Design (LLD)

 **Prepared For:** Academic / Project Submission